COESIONE ITALIA 21-27 PIEMONTE

Cofinanziato dall'Unione europea

REGIONE PIEMONTE

DOCENTE	Wolfgang Cecchin
UNITA' FORMATIVA	Linux Server
ARGOMENTO	Shell avanzata

Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

Shell avanzata

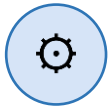
Reindirizzamento, processi e operatori

`fd 0/1/2` `2>&1` `&& ||` `$?` `nohup`

Cos'è un file descriptor



In Linux tutto è un file:
tastiera, schermo, rete, dispositivi



Un processo non comunica direttamente
con l'hardware



Chiede al kernel di aprire un canale →
riceve un numero intero: il fd



Il reindirizzamento cambia dove
punta quel numero, non il processo

I tre file descriptor standard

FD	Nome	Direzione	Default
0	`stdin`	ingresso ←	*tastiera*
1	`stdout`	uscita →	*terminale*
2	`stderr`	uscita →	*terminale*

⚠ stdout e stderr puntano entrambi al terminale, ma sono canali separati — questo permette di separare output normale ed errori.

Reindirizzamento: stdout e stderr

stdout (fd 1)

```
# sovrascrive
ls -l /etc > elenco.txt

# aggiunge in coda
ls -l /var/www >> elenco.txt

# equivalente esplicito
ls -l /etc 1> elenco.txt
```

stderr (fd 2)

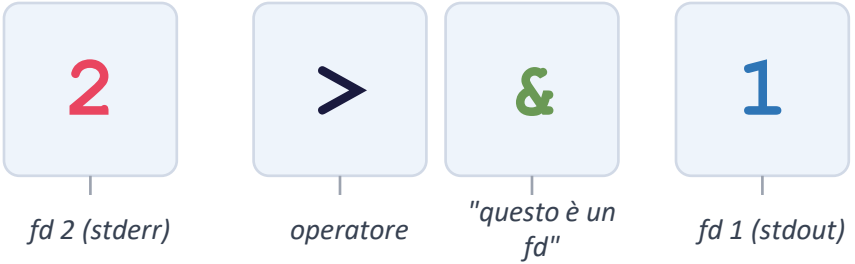
```
# separa errori dall'output
find / -name '*.conf' \
  > risultati.txt 2> errori.log

# scarta gli errori
find /var/www -name '*.php' \
  2> /dev/null

# grep: errori su file separato
grep -r 'password' /etc \
  2> /tmp/errori.log
```

> sovrascrive >> aggiunge 2> stderr /dev/null cestino

Reindirizzare entrambi: 2>&1



"Punta fd 2 verso lo stesso posto dove punta fd 1"

Senza & davanti all'1: il comando 2>1 scriverebbe stderr su un file chiamato letteralmente "1"

```
# Backup MySQL: tutto nello stesso log
mysqldump -u root -p fotodb \
  > /var/log/backup.log 2>&1

# Scarta tutto (silenzioso)
apt-get update &> /dev/null

# &> equivale a > ... 2>&1
# ma funziona solo in bash
```

2>&1 — l'ordine conta

La shell legge il reindirizzamento da sinistra a destra.

✓ CORRETTO

```
find / -name '*.conf' > tutto.log 2>&1

# passo 1 — > tutto.log:
#   fd1 → tutto.log
#   fd2 → terminale
# passo 2 — 2>&1:
#   fd2 → tutto.log  ✓
```

✗ SBAGLIATO

```
find / -name '*.conf' 2>&1 > tutto.log

# passo 1 — 2>&1:
#   fd2 → terminale  ← copia fd1 ora!
# passo 2 — > tutto.log:
#   fd1 → tutto.log
#   fd2 → terminale  ✗
```

Regola: prima sposta stdout con > poi aggancia stderr con 2>&1

Sintassi moderna (solo bash): `find / -name '*.conf' &> tutto.log`

stdin (fd 0) e /dev/null

stdin (fd 0) — leggere da file

```
# MySQL legge da file
# invece che dalla tastiera
mysql -u root -p fotodb < dump.sql

# Invia file come corpo mail
mail -s 'Report' \
  admin@studio.it \
  < /var/log/backup.log
```

/dev/null — il cestino

```
# Scarta solo stdout
apt-get update > /dev/null

# Scarta solo stderr
find / -name '*.conf' 2> /dev/null

# Scarta tutto
apt-get update &> /dev/null
```

Tutto quello che viene scritto in /dev/null scompare — nessun output, nessun errore visibile.

Operatori di controllo

`;`

Sempre in sequenza

Il secondo comando gira sempre, indipendentemente dall'esito del primo.

```
apt-get update ; apt-get upgrade
```

`&&`

Solo se successo

Il secondo comando gira solo se il primo ha restituito exit code 0.

```
mkdir /var/www/sito && cd /var/www/sito
```

`||`

Solo se fallimento

Il secondo comando gira solo se il primo ha restituito exit code ≠ 0.

```
cd /var/www/html || mkdir -p
/var/www/html
```

Combinazioni e negazione !

&& e || insieme simulano un if/else in una riga:

```
# Rete raggiungibile?
ping -c1 8.8.8.8 &>/dev/null && echo 'Rete OK' || echo 'Rete KO'

# nginx è attivo?
systemctl is-active nginx &>/dev/null \
  && echo 'nginx in esecuzione' || echo 'nginx fermo'

# Importa dump solo se il backup ha avuto successo
mysqldump -u root -p fotodb > backup.sql \
  && mysql -u root -p fotodb_test < backup.sql
```

! — nega il codice di uscita

```
# Crea il file di config solo se NON esiste già
[ ! -f /etc/nginx/conf.d/studio.conf ] && cp default.conf studio.conf
```

$? — il codice di uscita

Ogni comando restituisce un numero quando termina. È il valore che && e || controllano.

0	**successo**
1	errore generico
2	uso errato del comando
127	comando non trovato
130	interrotto con Ctrl+C

```
ls /etc
echo $?          # → 0

ls /inesistente
echo $?          # → 2

grep 'root' /etc/passwd
echo $?          # → 0 (trovato)

grep 'zzzz' /etc/passwd
echo $?          # → 1 (non trovato)
```

⚠ $? viene sovrascritto ad ogni comando, incluso echo — salvarlo subito in una variabile.

$? nella pratica — salvarlo subito

Dopo il comando che ci interessa, assegniamo $? a una variabile prima che venga sovrascritto:

```
mysqldump -u root -p fotodb > backup.sql
ESITO=$?

[ $ESITO -eq 0 ] && echo 'Backup OK' \
                 || echo "Backup fallito (codice: $ESITO)"

# Verifica connessione DB prima di procedere
mysql -u root -p -e 'SELECT 1;' &>/dev/null
ESITO=$?
[ $ESITO -ne 0 ] && { echo 'DB non raggiungibile'; exit 1; }
```

`[ $ESITO -eq 0 ]` → uguale a 0 `[ $ESITO -ne 0 ]` → diverso da 0

Processi in background

`cmd &`

Avvia in background

```
Il terminale è subito
libero
[1] 4821  ← job e PID
```

`jobs`

Lista job della sessione

```
[1]+ Running  cmd &
[2]- Stopped  vim file.txt
```

`fg %1`

Porta in foreground

```
fg   →  job corrente (+)
fg %2  →  job numero 2
```

`bg %1`

Riprendi in background

```
bg   →  job corrente
bg %2  →  job numero 2
```

`Ctrl+Z`

Sospendi foreground

```
[1]+ Stopped  ping ...
bg %1  per riprendere
```

`kill %1`

Termina job

```
SIGTERM (gentile)
kill -9 %1  →  SIGKILL
```

⚠ I job in background sono legati alla sessione corrente: chiudendo il terminale vengono terminati.

nohup — processi persistenti

nohup permette a un processo di sopravvivere alla chiusura del terminale, ignorando SIGHUP.

	`cmd &`	`nohup cmd &`
Terminale chiuso	processo terminato	**processo continua**
Output	sul terminale	**`nohup.out`**
Segnale SIGHUP	ricevuto	**ignorato**

grep -v grep filtra via la riga del grep stesso
(-v inverte il match: mostra le righe che NON contengono la stringa)

```
# Output su nohup.out
(default)
nohup php artisan queue:work &

# Con log personalizzato
nohup bash script.sh \
  > /var/log/script.log 2>&1 &

# Verifica esecuzione
ps aux | grep script.sh | grep
-v grep
```

Esercizio

Studio fotografico Luce & Scatto

Preparare lo script manutenzione.sh:

a) Creare /var/log/studio/ o uscire con messaggio di errore

b) Esportare la tabella sessioni su CSV, log errori, verificare $?

c) Trovare file .tmp: output su log, errori in append su errori.log

d) Avviare con nohup, output su /var/log/studio/manutenzione.log

e) Verificare con un solo comando che lo script sia in esecuzione

Riepilogo

```
# Reindirizzamento
>       stdout su file (sovrascrive)
>>      stdout su file (aggiunge)
2>      stderr su file
2>&1    stderr → dove punta stdout
&>      stdout + stderr (solo bash)
<       stdin da file

# Operatori
;       esegui sempre in sequenza
&&      solo se $? = 0
||      solo se $? ≠ 0
!       nega il codice di uscita
```

```
# $?
0       successo
≠ 0     errore
ESITO=$?    # salvare subito

# Background
cmd &       avvia in background
Ctrl+Z      sospendi
jobs        lista job sessione
fg %n       porta in foreground
bg %n       riprendi in background
kill %n     termina job

# nohup
nohup cmd > log 2>&1 &
```